

Data Mining

Data Preprocessing/Preparation-2

Aijun An

Department of Electrical Engineering and Computer Science
York University

Data Reduction Outline

- ▶ Feature Reduction
- ▶ Case Reduction

Case Reduction

- ▶ Objective: reduce the number of examples, the largest dimension in the data set
- ▶ How many examples are enough?
 - ▶ Application dependent - depends on the complexity of the patterns to be extracted from the data.
 - ▶ If the pattern is simple, the results are unlikely to change even with additional cases. For example, $x > 1$ completely separates two classes.
 - ▶ For complex patterns, large volumes of data can supply more evidence for the correctness of the induced patterns.
- ▶ Some types of problems requiring more data than others:
 - ▶ Multiclass classification
 - ▶ Regression
 - ▶ Imbalanced data sets
 - ▶ Deep neural network training

Case Reduction Methods

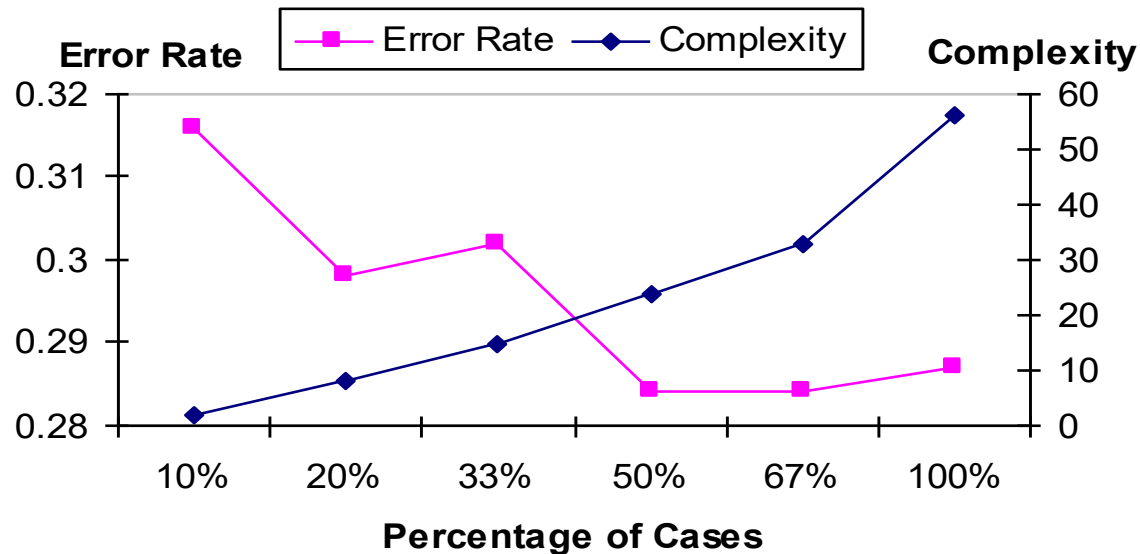
- ▶ Simple Random Sampling
 - ▶ A single sample
 - ▶ Incremental samples
 - ▶ Average samples
- ▶ Sampling by Boosting Class Balance
- ▶ Stratified Sampling

Single Simple Random Sample

- ▶ Choose n objects randomly from a set D of N objects ($n < N$) so that each object has the same probability of being chosen.
- ▶ Two methods
 - ▶ Simple random sampling without replacement (SRSWOR)
 - ▶ Each object cannot be chosen more than once
 - ▶ Simple random sampling with replacement (SRSWR)
 - ▶ Each time an object is drawn, it is recorded and placed back to D so that it can be drawn again.

Problem with Single Sampling

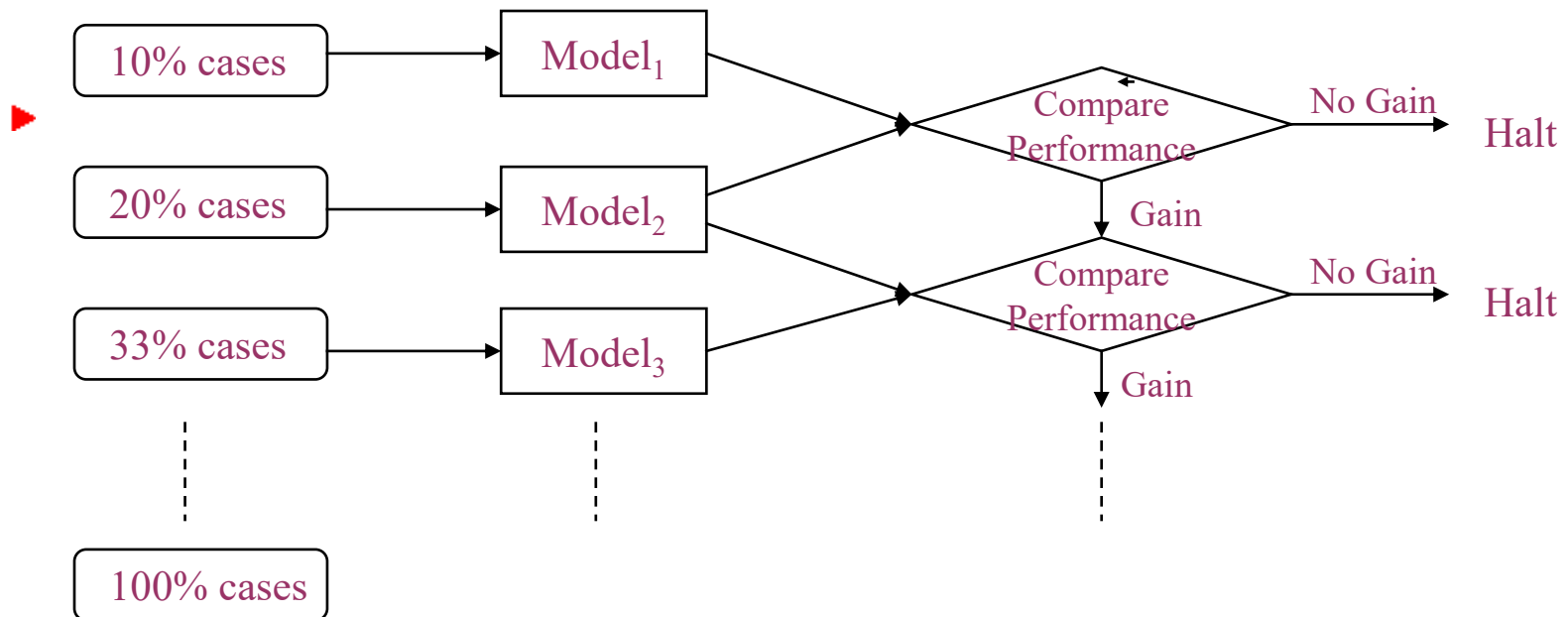
- ▶ We don't know the suitable sample size.
- ▶ Too small, the model may not be accurate enough; too big, the model may be more complex
 - ▶ An example:



Incremental Sampling

- ▶ Objective: Spot trends in error and complexity by *learning with incrementally larger random subsets of the data* to help produce a **single solution**.
- ▶ A typical pattern of incremental subsets is:

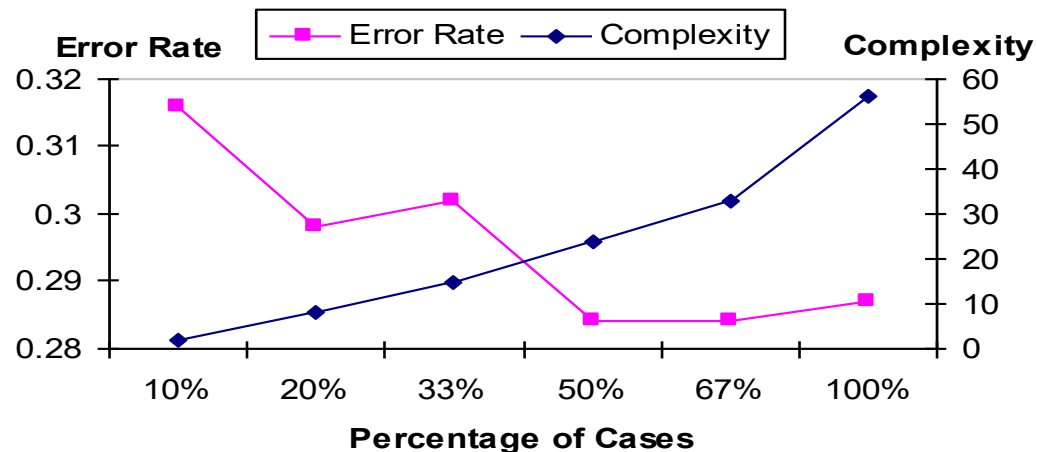
10%, 20%, 33%, 50%, 67%, 100%



Incremental Sampling (Cont.)

- ▶ Performance measures:
 - ▶ Error rate (test error, that is error on a test data set)
 - ▶ Complexity of the solution (e.g. number of nodes in a tree)
- ▶ Plot error and complexity relative to increasing sample size.

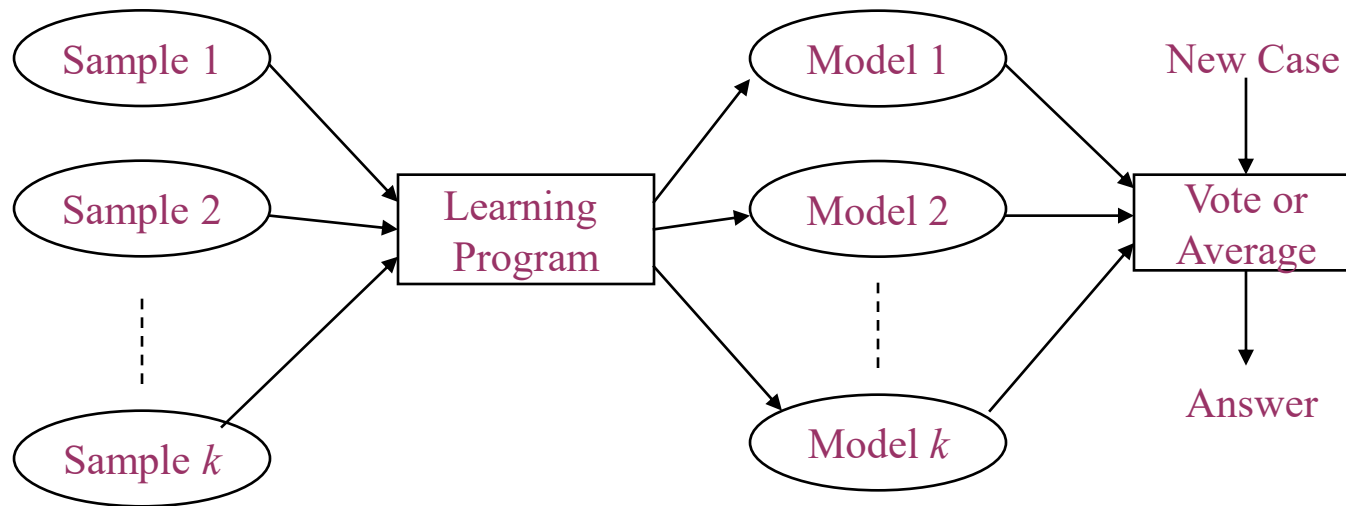
Example:



- ▶ Make decision on whether to do further sampling
 - ▶ Net changes in error and complexity are examined:
 - ▶ Is the error smaller?
 - ▶ Is the complexity acceptable?
 - ▶ Is complexity increasing much more than error is decreasing?

Average Sampling

- ▶ For a dataset containing a huge number of cases that exceed the maximum capacity of a learning program.
- ▶ Average sampling:
 - ▶ Select k random samples of n cases, where $n < N$.
 - ▶ Solutions from different samples are combined in the prediction phase.



- ▶ Averaged or voted solutions usually have less error than the single solution found on all cases in the database.

Sampling by Boosting Class Balance

- ▶ Suitable for **imbalanced data sets** where
 - ▶ almost all examples belong to the larger class, and
 - ▶ far fewer cases to the smaller, usually more interesting, class.
- ▶ Many real-world datasets are extremely imbalanced.
 - ▶ E.g., in a bio-chemistry data set for predicting biological potency of chemical compounds,
 - ▶ Only 0.16% of the compounds belong to the class of highly active compounds, which is the most interesting class that can lead to discovery of new drugs.
 - ▶ Remaining 99.84% of compounds are inactive.
- ▶ **Challenge:**
 - ▶ A model that classifies all examples into the larger class is very accurate
 - ▶ Needs to avoid learning such a model.

Sampling by Boosting Class Balance

Two ways to boost class balance:

- ▶ ***Up-sampling***: Increase the number of examples in the minority class
 - ▶ **Repeat** (or give higher weights to) the examples in the minority class in the sample - increase the sample size.
 - ▶ **Interpolation-based methods**
 - ▶ SMOTE: Generate synthetic data points for the minority class by linearly interpolating between minority instances and their neighbors.
 - ▶ ADSYN: an extension of SMOTE that generates additional synthetic samples for data points more difficult to learn based on the ratio of majority class samples in their nearest neighbors.

Sampling by Boosting Class Balance

- ▶ ***Down-sampling:***

- ▶ Keep the minority cases intact or randomly sample a high percentage of them, while including a low percentage random subset of a larger class.

- ▶ **Hybrid:** use both up-sampling and down-sampling

- ▶ Note:

- ▶ the predictive performance on the interesting minority cases may increase
- ▶ the overall predictive performance on data of all classes may decrease.
 - ▶ If consider the small class as positive, false positive rate increases

Stratified Sampling

- ▶ The data set D is partitioned into mutually disjoint subsets, called *strata*.
- ▶ Then randomly sample data from each stratum
- ▶ Objective: ensure a representative sample, especially when the data are skewed.

Outline

- ▶ Why preprocess the data?
- ▶ Data integration
- ▶ Data cleaning
- ▶ Data reduction
- ▶ Data augmentation
- ▶ Data labeling
- ▶ Data transformation
- ▶ Discretization

Data Augmentation

- ▶ Increase the size and diversity of data by artificially creating variations of the existing data
- ▶ Need for data augmentation
 - ▶ Deep learning often requires large amounts of data
 - ▶ Many applications have limited data available
 - ▶ Collecting large datasets (especially labeled data) is labor-intensive
 - ▶ More data can alleviate the overfitting problem, increasing predictive performance
 - ▶ Can be used to alleviate the class imbalance issue by augmenting data from the under-represented class

General Data Augmentation Techniques

▶ Basic Interpolation

- ▶ Interpolates existing examples (of the same class) to create new examples of the class.
- ▶ Local manipulation

▶ Augmentation data synthesis (Advanced)

- ▶ Learns data patterns from a global view and generates new examples with a learned model
- ▶ GANs (Generative Adversarial Networks)
 - ▶ Train a discriminator in conjunction with a generator, making the latter generate synthetic data that closely resembles the existing data.
- ▶ Variational Autoencoder
- ▶ Diffusion models

Data Augmentation Techniques for Images

- ▶ Special techniques for images
 - ▶ Geometric transformations
 - ▶ Randomly flip, crop, rotate, stretch, and zoom images
 - ▶ Color space transformations
 - ▶ Randomly change RGB color channels, contrast, and brightness
 - ▶ Kernel filters
 - ▶ Randomly change the sharpness or blurring of the images
 - ▶ Random erasing
 - ▶ Delete some parts of the initial image.
 - ▶ Mixing images
 - ▶ Blending and mixing multiple images

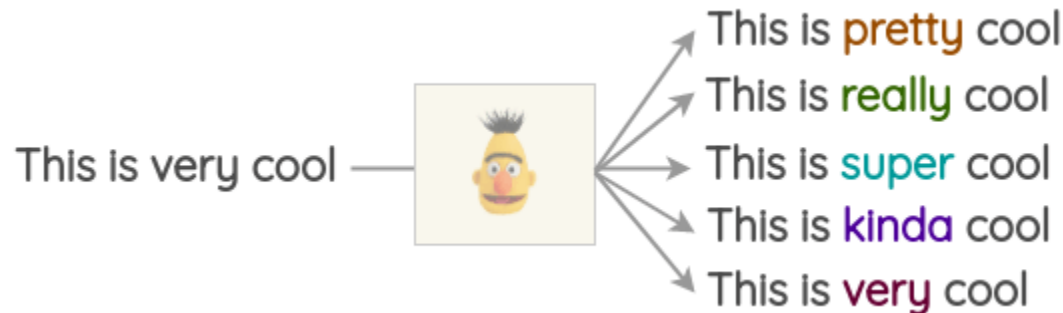
Data Augmentation Techniques for Texts

- ▶ Special basic techniques for texts

- ▶ Synonym replacement

- ▶ Lexical-based replacement

- ▶ Use a lexical database (e.g., WordNet) to find synonyms to replace words/tokens in the original sentence.



(Picture source: Amit Chaudhary "[A Visual Survey of Data Augmentation in NLP](#)")

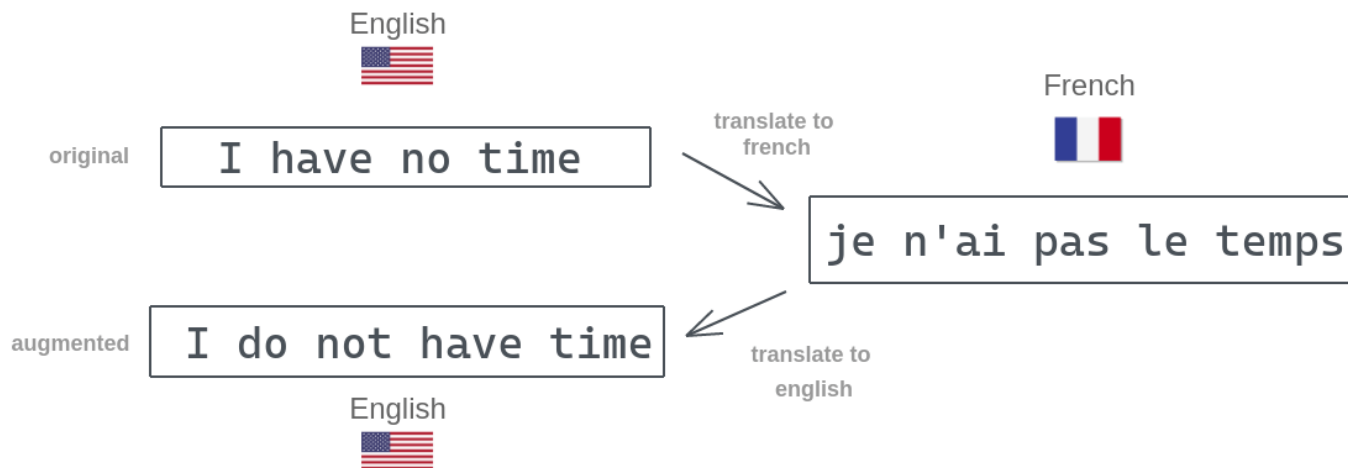
- ▶ Word embedding based replacement

- ▶ Use pre-trained word embeddings (e.g, GloVe, Word2Vec, fastText, ELMo, BERT, etc.) to find the nearest word vector as the replacement in the original sentence

Data Augmentation Techniques for Texts

► Back translation

- Translate a text into another language and then translate it back to the original language.
- Different sentences of the same or similar meaning are created.



(Picture source: Amit Chaudhary ["Back Translation for Text Augmentation with Google Sheets"](#))

Data Augmentation for Imbalanced Data

- ▶ Imbalanced data sets:
 - ▶ almost all examples belong to the larger class, and
 - ▶ far fewer cases to the smaller, usually more interesting, class.
- ▶ Many real-world datasets are extremely imbalanced.
 - ▶ E.g., in a bio-chemistry data set for predicting biological potency of chemical compounds,
 - ▶ Only 0.16% of the compounds belong to the class of highly active compounds, which is the most interesting class that can lead to discovery of new drugs.
 - ▶ Remaining 99.84% of compounds are inactive.
- ▶ Challenge:
 - ▶ A model that classifies all examples into the larger class is very accurate
 - ▶ Needs to avoid learning such a model.

Data Augmentation for Imbalanced Data

▶ Up-sampling

- ▶ Repeat (or give higher weights to) the examples in the minority class in the sample - increase the sample size.
- ▶ Interpolation-based methods
 - ▶ SMOTE: Generate synthetic data points for the minority class by linearly interpolating between minority instances and their neighbors.
 - ▶ ADSYN: an extension of SMOTE that generates additional synthetic samples for data points more difficult to learn based on the ratio of majority class samples in their nearest neighbors.

▶ Note:

- ▶ the predictive performance on the interesting minority cases may increase, but
- ▶ the overall predictive performance on data of all classes may decrease.

Outline

- ▶ Why preprocess the data?
- ▶ Data integration
- ▶ Data cleaning
- ▶ Data reduction
- ▶ Data augmentation
- ▶ Data labeling
- ▶ Data transformation
- ▶ Discretization

Data Labeling

- ▶ Provide correct outputs (e.g., class values) for training examples
- ▶ Essential for supervised training
- ▶ Methods
 - ▶ Crowdsourced labeling
 - ▶ Semi-supervised labeling
 - ▶ Active learning
 - ▶ Weakly-supervised labeling
 - ▶ Distant supervision

Crowdsourced Labeling

▶ Idea:

- ▶ Break down a labeling task into smaller and more manageable parts
- ▶ Outsource and distribute them to a large number of non-expert annotators

▶ Full human participation

▶ Issues

- ▶ Need clear guidelines
- ▶ Quality varies
 - ▶ Need to prune low-quality ones
- ▶ Inconsistency occurs
 - ▶ Resolve inconsistency or keep the consensus labels

Semi-Supervised Labeling

- ▶ Idea:
 - ▶ Leverage a small amount of labeled data to infer the labels of unlabeled data.
- ▶ Some methods
 - ▶ Multi-classifier based
 - ▶ Train multiple classifiers using the labeled data and use them to generate pseudo labels for unlabeled data.
 - ▶ Graph-based
 - ▶ Construct a graph where each node is an example and each edge represents the distance between 2 nodes
 - ▶ Infer labels through label propagation in the graph
 - ▶ Using reinforcement learning (RL) (e.g., ChatGPT)
 - ▶ Train initial models with labeled data
 - ▶ Train a reward function for RL from human feedback
 - ▶ Train a model using RL with the reward function

Active Learning

- ▶ An iterative procedure involving humans in the loop
- ▶ In each iteration:
 - ▶ Learn a model from the labeled data
 - ▶ Select an (or a batch of) unlabeled samples for humans to annotate
- ▶ Key in how to select the unlabeled samples in each iteration
 - ▶ Select the most uncertain ones (e.g., based on the predicted probability of an example belonging to a class)
- ▶ Needs continuous involvement from human

Weakly-Supervised Labeling

- ▶ Infer labels based on human-designed labeling functions.
- ▶ The function is often some heuristic rules and varies for different data types
 - ▶ For example, using seed words for text classification
 - ▶ Texts containing happy, joy, delighted, etc. are labeled as happy in emotion detection.
- ▶ Minimum or partial human participation
 - ▶ Provide the heuristics
- ▶ Can be used to quickly generate a large number of labels.

Distant Supervision

- ▶ A weakly-supervised approach
- ▶ Assign labels by leveraging external sources
 - ▶ For example, for relation extraction from two entities in text documents:
 - ▶ Semantic relationships between entities are labeled based on external data, such as Freebase.
- ▶ Often does not require human participation (automatic)
- ▶ Generated data can be noisy

Outline

- ▶ Why preprocess the data?
- ▶ Data integration
- ▶ Data cleaning
- ▶ Data reduction
- ▶ Data augmentation
- ▶ Data labeling
- ▶ Data transformation
- ▶ Discretization

Data Transformation

- ▶ Transform the data into an appropriate form for mining
- ▶ Attribute/feature construction
 - ▶ New attributes constructed from the given ones
 - ▶ e.g., compute average sale amount using total sale amount divided by units sold.
- ▶ Normalization: scale attribute values to fall within a small, specified range
 - ▶ min-max normalization
 - ▶ z-score normalization
 - ▶ normalization by decimal scaling
- ▶ Converting discrete attributes to numeric ones
- ▶ Discretization
 - ▶ Transform numeric attributes into discrete attributes

Data Transformation: Normalization

▶ min-max normalization

$$v' = \frac{v - \min_A}{\max_A - \min_A} (\text{new_max}_A - \text{new_min}_A) + \text{new_min}_A$$

where $\min_A$ and $\max_A$ are the minimum and maximum values of attribute A , and $[\text{new_min}_A, \text{new_max}_A]$ is the new range

▶ Example: Attribute *income* has values

- ▶ \$12,000, \$20,000, \$25,000, \$30,000, \$45,000, \$60,000, \$73,600, \$98,000

- ▶ normalized into values in range [0, 1]:

0, 0.093, 0.151, 0.209, 0.384, 0.558, 0.716, 1

▶ Problems:

- ▶ “Out of bounds” error occurs if a future input case falls outside the original range for A
- ▶ A too big or too small value could be noise. If they are used as min or max value for normalization, the results are not reliable.

Data Transformation: Normalization (Contd.)

- ▶ z-score normalization

$$v' = \frac{v - mean_A}{s_A}$$

where $mean_A$ is the mean of attribute A and s_A is the standard deviation of A (suppose values are : $v_1, v_2, \dots, v_n$):

$$s_A = \sqrt{\frac{1}{n} \sum_{i=1}^n (v_i - mean_A)^2} \quad \text{or} \quad s_A = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (v_i - mean_A)^2}$$

- ▶ Example:

- ▶ Attribute income has values:

- ▶ \$12,000, \$20,000, \$25,000, \$30,000, \$45,000, \$60,000, \$73,600, \$98,000

- ▶ The mean and standard deviation of the attribute *income* are

- ▶ \$45,450 and \$29,735, respectively

- ▶ With z-score normalization, the values are transformed into:

- ▶ -1.12, -0.86, -0.69, -0.52, -0.02, 0.49, 0.95, 1.77

- ▶ Advantages:

- ▶ useful when the actual min and max are unknown

- ▶ better deal with outliers than min-max normalization

Data Transformation: Normalization (Contd.)

- ▶ Normalization by decimal scaling

$$v_i' = \frac{v_i}{10^k}$$

where k is the smallest integer such that $\text{Max}(|v_i'|) \leq 1$

- ▶ Example:

- ▶ Suppose the recorded values of A range from -986 to 97
- ▶ The maximum absolute value of A is 986.
- ▶ Then $k = 3$
- ▶ -986 is normalized to -0.986 and 97 is normalized to 0.097

Data Transformation

- ▶ Transform the data into an appropriate form for mining
- ▶ Attribute/feature construction
 - ▶ New attributes constructed from the given ones
 - ▶ e.g., compute average sale amount using total sale amount divided by units sold.
- ▶ Normalization: scale attribute values to fall within a small, specified range
 - ▶ min-max normalization
 - ▶ z-score normalization
 - ▶ normalization by decimal scaling
- ▶ Converting discrete attributes to numeric ones
- ▶ Discretization
 - ▶ Transform numeric attributes into discrete attributes

Converting Discrete Attributes to Numeric Ones

- ▶ Some popular machine learning algorithms (such as neural networks, SVM, etc.) cannot deal with categorical attributes
- ▶ Method 1:
 - ▶ Convert a nominal attribute to a numerical attribute, for example:
 - ▶ $\text{yes} \Rightarrow 1$ and $\text{no} \Rightarrow 0$
 - ▶ $\text{blue} \Rightarrow 1$, $\text{yellow} \Rightarrow 2$, $\text{red} \Rightarrow 3$, etc.
 - ▶ Problem?
- ▶ Method 2:
 - ▶ Convert a nominal attribute to n binary attributes, where n is the number of values of the nominal attribute
 - ▶ For example, if the domain of attribute A is $\{a_1, a_2, a_3\}$, we convert it into 3 attributes to represent A.
 - ▶ If $A=a_1$, the values of the three attribute are 1, 0, 0.
 - ▶ If $A=a_2$, the values of the three attribute are 0, 1, 0.
 - ▶ If $A=a_3$, the values of the three attribute are 0, 0, 1.

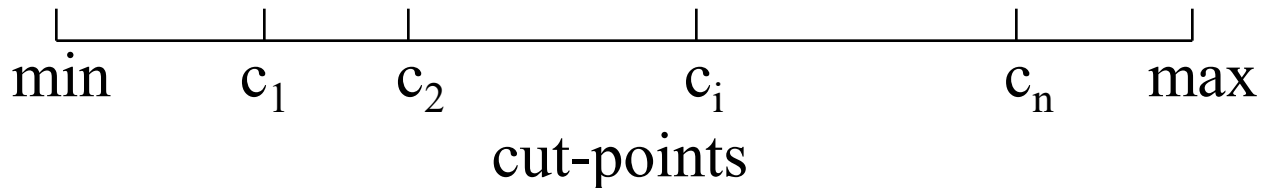
*One-hot
encoding*

Outline

- ▶ Why preprocess the data?
- ▶ Data integration
- ▶ Data cleaning
- ▶ Data reduction
- ▶ Data augmentation
- ▶ Data labeling
- ▶ Data transformation
- ▶ Discretization

What is Discretization?

- ▶ A discretization algorithm
 - ▶ converts continuous attributes into discrete attributes by partitioning the range of a continuous attribute into intervals.
 - ▶ Interval labels can then be used to replace actual data values.

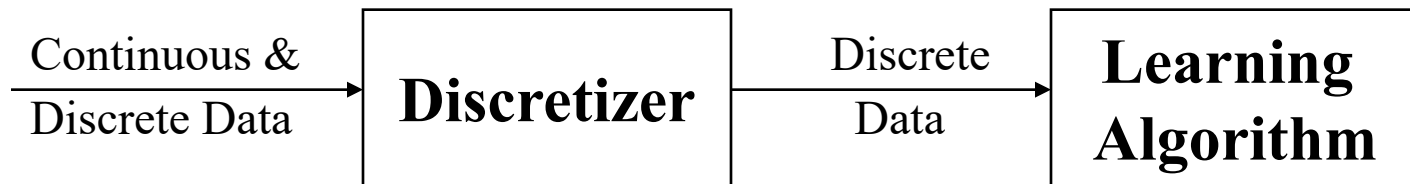


Why Need Discretization?

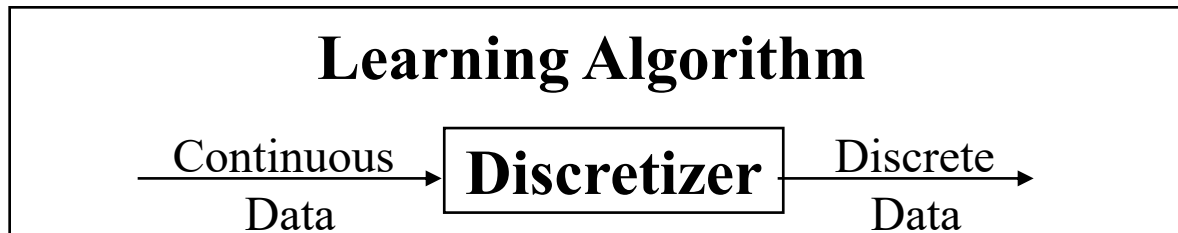
- ▶ Some learning algorithms are limited to discrete inputs.
- ▶ Efficiency: handling (lots of) continuous values tends to slow down learning considerably. (*Unique Value reduction*)
- ▶ Accuracy: in the presence of noise good discretization can sometimes improve predictive accuracy. (*Smoothing out noise*)
- ▶ Intelligibility: discretization may lead to smaller sizes of induced trees or rule sets.

Two Architectures

- ▶ Discretization before learning starts (Static discretization)

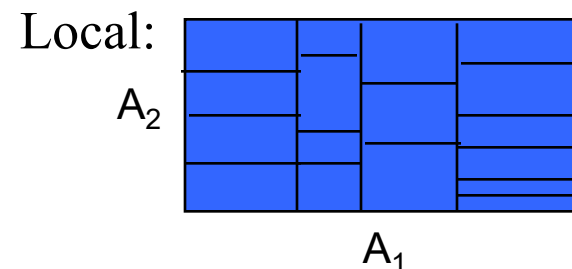
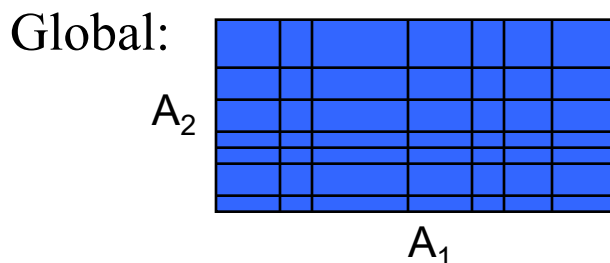


- ▶ Discretization during the learning process (Dynamic discretization)



Classification of Discretization Methods

- ▶ Supervised vs. unsupervised.
 - ▶ Supervised discretization uses class information.
 - ▶ Unsupervised does not use class labels.
- ▶ Bottom-up vs. top-down
 - ▶ Bottom-up: start from intervals with one value each and repeatedly merge intervals until some stopping criterion is satisfied.
 - ▶ Top-down: start from one interval with all values and repeatedly split intervals until some stopping criterion is satisfied.
- ▶ Global vs. local
 - ▶ Global: an attribute is partitioned over the entire continuous range, using global information and independent of other attributes.
 - ▶ Local: partition is applied to local regions of an attribute range.



Unsupervised Discretization

▶ Equal-width binning

- ▶ It divides the range of an attribute into k intervals of equal size: uniform grid
- ▶ If A and B are the lowest and highest values of the attribute, the width of intervals will be: $W = (B - A)/k$.

▶ Equal-depth/frequency binning

- ▶ It divides the range into k intervals, each containing approximately the same number of values

▶ k-means clustering

- ▶ Given k bins, distribute the values in the bins to minimize the average distance of a value from its bin mean.

Equal-width Binning

- Sorted data for price (in dollars): 4, 8, 9, 15, 21, 21, 24, 25, 26, 28, 29, 34
- Partition into (equi-width) bins: 3 intervals of equal size
 - Bin 1: 4, 8, 9
 - Bin 2: 15, 21, 21, 24
 - Bin 3: 25, 26, 28, 29, 34
- Pros and Cons
 - ▶ The most straightforward
 - ▶ But outliers may dominate presentation
 - ▶ Skewed data is not handled well.

Equal-depth Binning

- Sorted data for price (in dollars): 4, 8, 9, 15, 21, 21, 24, 25, 26, 28, 29, 34 (12 points in total)
- Partition into 3 (equi-depth) bins:
 - Bin 1: 4, 8, 9, 15
 - Bin 2: 21, 21, 24, 25
 - Bin 3: 26, 28, 29, 34
- ⑩ Good data scaling; better handle skewed data

K-mean Clustering

- ▶ Input: (1) a set of values for an attribute
(2) k = number of bins
- ▶ Sort the input values and keep the unique values
- ▶ Create k bins using equal-depth binning
- ▶ Compute bin means ($mean_1, mean_2, \dots, mean_k$)
- ▶ Compute global distance: $D_{new} = \sum_i \sum_j (v_{ij} - mean_i)^2$
where $mean_i$ is the mean in bin_i and v_{ij} is the j th value in bin_i .
- ▶ Repeat
 - ▶ $D_{old} = D_{new}$
 - ▶ for each bin_i
 - ▶ for each v_{ij} in bin_i
 - ▶ If $(|v_{ij} - mean_{i-1}|) < (|v_{ij} - mean_i|)$, move v_{ij} to bin_{i-1} .
 - ▶ If $(|v_{ij} - mean_{i+1}|) < (|v_{ij} - mean_i|)$, move v_{ij} to bin_{i+1} .
 - ▶ Compute new bin means and D_{new}
- ▶ Until D_{new} is not less than D_{old} .

Supervised Discretization

▶ ChiMerge

- ▶ Based on chi-square test
- ▶ Entropy-based discretization method
 - ▶ Based on an entropy minimization heuristic

Entropy-Based Discretization

- ▶ Supervised, top-down discretization
- ▶ Employs an entropy minimization heuristic for splitting the range of a continuous attribute.
- ▶ Given a set S of examples and k classes, the *entropy* of S with respect to the k classes is defined as:

$$Ent(S) = -\sum_{i=1}^k P(C_i) \log_2(P(C_i))$$

where $P(C_i)$ is the probability of examples in S that belong to C_i .

- ▶ The bigger $Ent(S)$ is, the more impure S is.

Entropy-Based Discretization

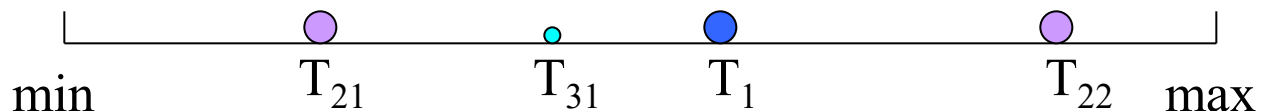
Given an attribute A and a set S of training examples:

- ▶ Sort the examples in a set S by increasing values of the attribute A : $\{v_1, v_2, \dots, v_n\}$.
- ▶ A potential cut-point T : midpoint between v_i and v_{i+1} dividing S into S_1 : $\{v_1, v_2, \dots, v_i\}$ and S_2 : $\{v_{i+1}, \dots, v_n\}$.
- ▶ A total of $n-1$ potential cut-points.
- ▶ Suppose a cut-point T partitions S into S_1 and S_2 . Entropy (with respect to the class attribute) after the partition induced by cutpoint T :

$$Ent(T, S) = \frac{|S_1|}{|S|} Ent(S_1) + \frac{|S_2|}{|S|} Ent(S_2)$$

where $|S|$, $|S_1|$ and $|S_2|$ = # of examples in S , S_1 and S_2

- ▶ Select T_A for which $E(T_A, S)$ is **minimal** to split the range into two subranges
- ▶ The process is recursively applied to partitions obtained until some stopping criterion is met.



Stopping Criteria for Entropy-Based Discretization

▶ Stopping criteria in D-2 (Catlett, 1991):

Recursive partitioning stops if any of the following is satisfied:

- ▶ all the examples in the interval belong to the same class.
- ▶ number of examples in an interval is below a given level;
- ▶ maximum number of cut-points for an attribute is reached;
- ▶ the entropy reduction on all possible cut-points is equal;

▶ Stopping criterion based on Minimum Description Length Principle (MDLP) (Fayyad and Irani, 1993):

Recursive partitioning stops iff

$$Ent(S) - Ent(T, S) \leq \frac{\log_2(N-1)}{N} + \frac{\Delta(T; S)}{N}$$

$$\Delta(T; S) = \log_2(3^k - 2) - [kEnt(S) - k_1Ent(S_1) - k_2Ent(S_2)]$$

where T is the cut point leading to the smallest $Ent(T, S)$, k , k_1 and k_2 are the number of classes in S , S_1 and S_2 , respectively, and N is the number of examples in S .

Summary

- ▶ Data preparation is a big issue for data mining
- ▶ Data preparation includes
 - ▶ Data integration
 - ▶ Data cleaning
 - ▶ Handle missing values
 - ▶ Detect and remove noise
 - ▶ Data reduction
 - ▶ Data augmentation
 - ▶ Data labeling
 - ▶ Data transformation
 - ▶ Discretization
- ▶ A lot of methods have been developed but still an active area of research